

CCOP v1.0 — 폐쇄망 방문설치 런북 (Rocky 10 · RTX 6000 Ada ×8 · USB 반입)

USB로 반입해 인터넷 없는 폐쇄망 VM에 CCOP를 설치·검증하기 위한 **현장 실행 런북**. 1차(인프라)와 2차(GPU+Text2Cypher)를 분리하여, 방문설치 현장 리스크가 큰 GPU 스택(드라이버·Secure Boot·CUDA↔vLLM 버전정합)을 인프라 안정화 이후로 미룬다.

폐쇄망 일반 원리는 `AIRGAP_DEPLOY_GUIDE.md`, 인터넷 운영 VM 런북은 `VM_DEPLOY_OPERATIONS_GUIDE.md` 참고.

최종 갱신: 2026-07-14 (번들 자동화·vLLM 태그 확정 + 현장 DB 선설치 반영)

 **2026-07-14 현장 작업일지 접수**: 대상 폐쇄망 VM에 AgensGraph 16.9가 네이티브(베어메탈)로 선설치됨 (포트 5333, Rocky 10.1, DBA 튜닝 완료 — `deploy/AIRGAP_SITE_DB_LOG_20260714.md`). 이에 따라 두 시나리오로 분기한다:

시나리오	DB	compose 파일	비고
A (현장 기본)	네이티브 AgensGraph 16.9 (호스트 :5333)	<code>docker-</code> <code>compose.airgap.nativedb.yml</code>	DB 컨테이너·이미지 불필요. 호환 검증 V1~V5 필수 (preflight \$5)
B (폴백)	컨테이너 (bitnine/agensgraph:v2.13.2)	<code>docker-compose.airgap.yml</code>	A 의 호환 검증 실패 시 즉시 전환 — agensgraph 이미지는 보험으로 번들에 유지

0. 대상 환경 & 단계 전략

0.1 대상 폐쇄망 VM

항목	사양
OS	Rocky Linux 10 (기존 운영 VM은 9.6 — 버전 상향)
CPU / MEM	vCPU 64 / 128 GB
Disk / NIC	630 GB / 10G ×1
GPU	NVIDIA RTX 6000 Ada ×8 (48GB ECC ×8 = 384GB, NVLink 미지원)
반입	USB 방문 설치 (인터넷·외부망 전면 차단)
목적	SKAI2_VM 운영 서비스를 폐쇄망에 이식해 테스트

0.2 이 저장소에 준비된 폐쇄망 전용 자산

방문설치용 파일은 이미 소스에 포함되어 있다(소스 반입 시 함께 들어감). 폐쇄망에서 새로 편집할 필요 없다.

파일	용도
<code>docker-compose.airgap.yml</code>	폐쇄망 전용 컴포즈. app은 <code>image: ccop_app:1.0</code> (빌드 안 함), sllm은 <code>profile: gpu</code> (2차만)

파일	용도
deploy/.env.airgap.phase1.template	1차 환경변수 템플릿 (LLM 미설정)
scripts/gen_selfsigned_cert.sh	nginx용 자체서명 인증서(cert.pem / key.pem) 생성
scripts/build_airgap_bundle.sh	1차 번들 자동 생성기 — §2 전 과정(이미지 빌드/save-rpm 수집·DB 덤프·인증서·소스 tar·체크섬) 자동화
Dockerfile.airgap + requirements.airgap.txt	폐쇄망 슬림 앱 이미지(RAG/torch 미포함, ~1GB) — 번들 스크립트가 존재 시 자동 사용
docker-compose.airgap.nativedb.yml	시나리오 A(네이티브 DB) 전용 컴포즈 — app+nginx(+sllm profile)만, host-gateway 로 호스트 :5333 접속
deploy/AIRGAP_SITE_DB_LOG_20260714.md	현장 DB 선설치 작업일지 정리 + 신규 검증 항목 V1~V5

⚠ 기존 docker-compose.cslee.yml 은 인터넷 운영(skai2_vm)용이므로 폐쇄망에서 쓰지 않는다.

0.3 1차 / 2차 분리 원칙

폐쇄망에서는 docker pull · pip install · dnf (온라인)·OpenAI·HuggingFace·GitHub가 모두 불가하다. 모든 의존성은 인터넷 되는 staging(x86-64) 에서 번들로 만들어 USB로 반입한다.

앱 코드 확인 결과 LLM(vLLM/OpenAI) 없이도 앱은 정상 기동한다 (AIService 가 전부 @staticmethod + lazy client → import/기동 시 LLM 클라이언트를 만들지 않음; config.py 는 키가 없으면 None 으로 로드). 따라서 인프라를 먼저 세우고 GPU/T2C를 나중에 붙인다.

단계	범위	현장 리스크
1차	Docker + AgensGraph + Flask앱 + nginx + 실데이터	낮음 (컨테이너가 OS차 흡수)
2차	NVIDIA 드라이버 + container-toolkit + vLLM + T2C 모델	높음 (드라이버/Secure Boot/버전정합)

0.4 단계별 기능 범위

✅ 1차로 검증 가능	ⓘ 2차로 유예
앱 기동 · /health · nginx HTTPS	자연어 → Cypher 자동변환 (Text2Cypher) — 딱 이것 1개
그래프 목록/검색/확장/경로탐색	
그래프 시각화 UI · 모델러(수동 노드/엣지)	
직접 Cypher 입력 실행 · 외부기관 조회 API	

1차 .env 에 SLLM_ENDPOINT · OPENAI_API_KEY 를 비워두면 자연어 질의 엔드포인트만 실패하고 나머지는 전부 정상. 앱 이미지 재빌드 없이 2차에서 .env 세 줄(+profile) 만으로 T2C가 활성화된다.

1. 방문 전 사전 확인 (D-day 이전)

- [] staging 머신 = linux/amd64 확보 (Rocky 10 권장). ⚠️ Apple Silicon 맥에서 빌드 금지 — `docker save` 이미지에 arm64가 박혀 x86 VM에서 실행 실패.
- [] 기관 보안 절차 확인: 매체 반입 신청·사전 승인·백신 검사·매체 반출입 대장 양식.
- [] USB 준비: exFAT 포맷 (FAT32 금지 — 단일 파일 4GB 제한). 1차 8GB↑ / 2차 64GB↑.
- [] [2차 대비] GPU-VM 관계 확인: 8장이 이 VM에 passthrough되는가, 아니면 별도 GPU 노드인가? (후자면 `SLLM_ENDPOINT` 를 GPU 노드 내부 IP로)
- [] [2차 대비] BIOS Secure Boot 상태 확인 (ON이면 NVIDIA 커널 모듈 MOK 서명 필요).
- [] [2차 대비] 드라이버↔vLLM CUDA 버전 조합을 staging GPU에서 1회 검증 후 pin.

【1차】인프라 배포 — GPU / Text2Cypher 제외

2. staging에서 1차 번들 준비 (인터넷 O · linux/amd64)

작업 루트: `~/ccop_bundle_p1/`

자동화: §2.1~2.6 전 과정은 스크립트 하나로 대체된다 (아래 수동 절차는 이해·복구용 레퍼런스).

```
bash PGPASSWORD='<db비번>' bash scripts/build_airgap_bundle.sh # --skip-db --skip-rpm --out DIR (-
-help 참고)
```

앱 이미지는 `Dockerfile.airgap` (슬림 ~1GB)이 있으면 **자동 선택**된다.

2.1 앱 이미지 빌드 → save

```
cd ccop_test # 소스 체크아웃 (docker-compose.airgap.yml 등 폐쇄망 자산 포함)
mkdir -p ~/ccop_bundle_p1/{images,rpms,db,src}
docker build --platform linux/amd64 -t ccop_app:1.0 -f Dockerfile .
docker save ccop_app:1.0 -o ~/ccop_bundle_p1/images/ccop_app_1.0.tar
```

2.2 베이스 이미지 pull → save (1차는 vLLM 제외)

```
docker pull bitnine/agensgraph:v2.13.2
docker pull nginx:alpine
docker save bitnine/agensgraph:v2.13.2 -o ~/ccop_bundle_p1/images/agensgraph_2.13.2.tar
docker save nginx:alpine -o ~/ccop_bundle_p1/images/nginx_alpine.tar
```

2.3 Docker CE el10 오프라인 rpm 수집

```
sudo dnf config-manager --add-repo https://download.docker.com/linux/rhel/docker-ce.repo
dnf download --resolve --alldeps --destdir=~/ccop_bundle_p1/rpms \
    docker-ce docker-ce-cli containerd.io docker-compose-plugin
```

2.4 DB 덤프 (테스트용 경량 그래프)

```
pg_dump -h 49.50.128.28 -p 5333 -U <user> -d tccopdb -F c \
    -f ~/ccop_bundle_p1/db/tccopdb.dump
gzip ~/ccop_bundle_p1/db/tccopdb.dump
```

⚠ 테스트엔 `tccop_graph_v6` (149노드)만 있으면 충분. `osint_ontology` (6.89M노드)가 포함되면 덤프가 수 GB로 커진다. 경량 그래프만 담은 별도 DB를 덤프하거나, 복원 후 폐쇄망에서 불필요 그래프를 정리한다.

2.5 소스 번들 + 자체서명 인증서

폐쇄망 자산(`docker-compose.airgap.yml`, `deploy/.env.airgap.phase1.template`, `scripts/gen_selfsigned_cert.sh`, `nginx.cslee.conf`, `init_db.sh`)은 소스에 이미 포함되어 있으므로 소스 번들 하나면 된다.

```
# 소스 (git bundle 또는 tar)
git bundle create ~/ccop_bundle_p1/src/ccop_test.bundle --all
# 또는: tar czf ~/ccop_bundle_p1/src/ccop_test.tar.gz --exclude='.git' .

# 자체서명 인증서 생성 (deploy/ssl/cert.pem, key.pem) - 소스에 포함시켜 반입
bash scripts/gen_selfsigned_cert.sh ccop.cslee.internal 3650
# → 생성된 deploy/ssl/ 이 소스 tar/bundle 에 포함되도록 인증서 생성 후 번들링
```

인증서 파일명 `cert.pem` / `key.pem` 은 `nginx.cslee.conf` 의 `ssl_certificate /etc/nginx/ssl/cert.pem;` 와 이미 일치한다(스크립트가 그 이름으로 생성). CN은 nginx `server_name` (기본 `ccop.cslee.internal`)과 맞추면 좋다.

2.6 체크섬 + USB 적재

```
cd ~/ccop_bundle_p1
find . -type f ! -name SHA256SUMS -exec sha256sum {} \; > SHA256SUMS
cp -rv ~/ccop_bundle_p1 /run/media/$USER/USB/ # exFAT USB
```

1차 번들 (~4-5GB, 8GB USB로 충분):

```
ccop_bundle_p1/
├─ images/    ccop_app_1.0.tar · agensgraph_2.13.2.tar · nginx_alpine.tar
├─ rpms/      docker-ce 등 el10 오프라인 rpm
├─ db/        tccopdb.dump.gz
├─ src/       ccop_test.bundle (airgap 자산·인증서 포함)
└─ SHA256SUMS
```

3. USB 반입 (현장 보안 절차)

- [] 매체 반입 신청서 제출 + 사전 승인
- [] 반입 게이트 백신 검사 통과
- [] 매체 반출입 대장 기록 (일련번호·용량·용도)
- [] 폐쇄망 VM에서 번들 복사 후 무결성 검증

4. 폐쇄망 VM 오프라인 설치 (1차)

4.1 무결성 검증

```
sudo mkdir -p /opt/ccop_bundle && cp -r /run/media/$USER/USB/ccop_bundle_p1/* /opt/ccop_bundle/  
cd /opt/ccop_bundle  
sha256sum -c SHA256SUMS          # 전 파일 OK 확인
```

4.2 Docker 오프라인 설치

```
sudo dnf install -y ./rpms/*.rpm  
sudo systemctl enable --now docker  
docker version
```

Rocky 10 기본은 Podman이지만, 기존 compose를 그대로 쓰기 위해 Docker CE를 반입·설치한다.

4.3 이미지 load

```
for t in images/*.tar; do sudo docker load -i "$t"; done  
docker images          # ccop_app:1.0 · bitnine/agensgraph · nginx:alpine 확인
```

4.4 소스 복원 + .env 작성 (compose 편집 불필요)

```
sudo mkdir -p /opt/ccop  
git clone /opt/ccop_bundle/src/ccop_test.bundle /opt/ccop      # bundle 일 때  
# 또는: tar xzf /opt/ccop_bundle/src/ccop_test.tar.gz -C /opt/ccop  
cd /opt/ccop  
  
# 1차 .env 작성 (템플릿 복사 후 비밀번호/도메인만 채움)  
cp deploy/.env.airgap.phase1.template .env  
vi .env          # DB_PASSWORD, SECRET_KEY, ADMIN_PASSWORD, CORS_ORIGINS 입력 (SLLM_* 는 1차에 비움)  
  
# 인증서 확인 (번들에 없으면 여기서 생성)  
ls deploy/ssl/cert.pem deploy/ssl/key.pem || bash scripts/gen_selfsigned_cert.sh ccop.cslee.internal 3650
```

`docker-compose.airgap.yml` 은 app이 이미 `image: ccop_app:1.0` 이고 sllm이 `profile: gpu` 라 1차엔 편집 없이 그대로 사용한다.

4.5 DB 준비 + 복원

시나리오 A — 네이티브 AgensGraph 16.9 (현장 기본)

```
# DBA 협의/확인 (필수): ① pg_isready -p 5333 ② pg_hba.conf 에 docker 브리지 대역 허용
# ③ 앱 전용 DB/계정: createdb -p 5333 tccopdb + CREATE USER ccop ... OWNER 지정
gunzip -k /opt/ccop_bundle/db/tccopdb.dump.gz
pg_restore -h localhost -p 5333 -U hlucyber -d tccopdb --clean --if-exists \
    /opt/ccop_bundle/db/tccopdb.dump # ⚠ V3 검증: 구버전(2.13) 덤프 → 16.9 복원
psql -p 5333 -U ccop -d tccopdb -c "SELECT 1" # 앱 계정 접속 확인
# 복원 실패(카탈로그 비호환) 시 → 플랜 B: 원천 CSV 반입분을 앱 ETL 로 재적재 (preflight $5 V3)
```

시나리오 B — 컨테이너 DB (플백)

```
docker compose -f docker-compose.airgap.yml up -d agensgraph
docker compose -f docker-compose.airgap.yml logs -f agensgraph # "initialization complete" (init_db.sh 가 확

gunzip -k /opt/ccop_bundle/db/tccopdb.dump.gz
docker cp /opt/ccop_bundle/db/tccopdb.dump ccop_agensgraph:/tmp/dump.dump
docker exec -it ccop_agensgraph pg_restore -U ccop -d tccopdb --clean --if-exists /tmp/dump.dump

docker exec -it ccop_agensgraph psql -U ccop -d tccopdb \
    -c "LOAD 'age'; SET search_path=ag_catalog; SELECT name FROM ag_graph;"
```

4.6 app + nginx 기동

```
# 시나리오 A (네이티브 DB):
docker compose -f docker-compose.airgap.nativedb.yml up -d # app + nginx
docker compose -f docker-compose.airgap.nativedb.yml ps # 전부 healthy

# 시나리오 B (플백):
docker compose -f docker-compose.airgap.yml up -d # agensgraph + app + nginx
```

4.7 1차 검증 체크리스트

- [] `curl -sk https://localhost/api/v1/health` → `{"status":"healthy"}`
- [] `curl -sk https://localhost/api/v1/graph/list` → 빈 그래프 아닌 **실데이터** 반환
- [] 브라우저(내부망) → `https://<VM_IP>/` UI 로딩
- [] 그래프 검색 / 노드 확장 / 경로탐색 동작
- [] 직접 Cypher 입력 실행 → 결과 반환
- [] 모델러에서 수동 노드/엣지 생성·편집
- [] (선택 — 법률 RAG 포함 소스일 때) `docker exec ccop_app python scripts/ingest_legal_corpus.py --no-embed` → `/api/v1/legal/status` 에서 BM25-only 적재 확인

- [] 자연어 질의창은 **의도적으로 미동작** (2차 예정 — 운영자 안내)
- [] 전 과정 **외부 호출 0건** (폐쇄망 내부 완결)

→ 1차 완료. 인프라·데이터·시각화 검증 끝. 여기까지 안정화한 뒤 2차 진행.

【2차】GPU + Text2Cypher 추가 — 인프라 안정화 후

5. 2차 번들 준비 (staging · linux/amd64)

```
mkdir -p ~/ccop_bundle_p2/{images,model,nvidia}

# vLLM 이미지 - 태그 고정 (latest 금지). 권장: v0.6.3.post1
#   근거: 운영 검증 서버 레시피(vllm 0.6.3.post1 + transformers 4.46.x + --chat-template 명시)와
#   동일 버전. cu121 빌드라 R530+ 드라이버(폐쇄망 신규 드라이버 포함)에서 동작.
#   다른 태그를 쓰려면 staging GPU에서 모델 로드+추론 1회 검증 후 반입할 것.
docker pull vllm/vllm-openai:v0.6.3.post1
docker save vllm/vllm-openai:v0.6.3.post1 -o ~/ccop_bundle_p2/images/vllm_openai.tar

# T2C 모델 풀웨이트 (~15GB)
rsync -av ai-kyw-dev@192.168.1.133:.../qwen25_t2c_v42_v1_merged/ \
    ~/ccop_bundle_p2/model/qwen25-t2c-v42/

#   포함 확인: 4 샤드 + config + tokenizer + chat_template.jinja

# NVIDIA 드라이버 local-repo rpm (Rocky 10 = CUDA repo 표준 rpm; 9/8 의 dnf module 과 다름)
wget https://developer.download.nvidia.com/compute/nvidia-driver/<ver>/local_installers/\
nvidia-driver-local-repo-rhel10-<ver>.x86_64.rpm -P ~/ccop_bundle_p2/nvidia/
dnf download --resolve --alldeps --destdir=~/ccop_bundle_p2/nvidia nvidia-container-toolkit

cd ~/ccop_bundle_p2 && find . -type f ! -name SHA256SUMS -exec sha256sum {} \; > SHA256SUMS
# exFAT USB 적재 (2차 ~30GB → 64GB USB)
```

6. 폐쇄망 GPU 스택 설치

```
cd /opt/ccop_bundle_p2 && sha256sum -c SHA256SUMS
```

6.1 드라이버

```
sudo dnf install -y ./nvidia/nvidia-driver-local-repo-rhel10-*.rpm
```

```
sudo dnf install -y nvidia-driver cuda-drivers
```

(Secure Boot ON이면 MOK 등록 후 재부팅하여 enroll)

6.2 container-toolkit + docker 런타임 연결

```
sudo dnf install -y ./nvidia/*nvidia-container*.rpm ./nvidia/*libnvidia-container*.rpm
```

```
sudo nvidia-ctk runtime configure --runtime=docker
```

```
sudo systemctl restart docker
```

6.3 검증

```
nvidia-smi # RTX 6000 Ada 8장 인식
```

```
docker run --rm --gpus all <cuda이미지> nvidia-smi # 컨테이너 GPU 접근
```

⚠ **주의:** ① Secure Boot ON → 모듈 미서명 시 `nvidia-smi` 실패 → MOK 등록. ② vLLM 이미지 CUDA > 드라이버 지원 CUDA면 기동 실패 → 태그 재확인. ③ RTX 6000 Ada는 **sm_89 · FP8** 지원, vLLM 완전 호환.

7. vLLM 서버 + Text2Cypher 활성화

7.1 이미지·모델 적재

```
docker load -i /opt/ccop_bundle_p2/images/vllm_openai.tar
sudo mkdir -p /opt/ccop/model && cp -r /opt/ccop_bundle_p2/model/qwen25-t2c-v42 /opt/ccop/model/
```

7.2 .env 세 줄 추가 + 기동 (compose 편집·이미지 재빌드 없음)

.env 에서 아래 세 줄 주석 해제(값 입력):

```
SLLM_ENDPOINT=http://sllm:8000/v1
SLLM_MODEL_NAME=qwen25-t2c-v42
VLLM_TAG=<위에서 반입한 vllm 이미지 태그>
```

```
cd /opt/ccop
docker compose -f docker-compose.airgap.yml --profile gpu up -d      # sllm 기동 (⚠ TP=1, 1장)
docker compose -f docker-compose.airgap.yml up -d app                # .env 재반영 (app 재기동)
# vLLM 응답 확인 (app 컨테이너에서 내부망 통신)
docker exec -it ccop_app python -c "import urllib.request;print(urllib.request.urlopen('http://sllm:8000/v1/m
```

7.3 ✅ 2차 검증 (e2e)

- [] 위 /v1/models → qwen25-t2c-v42 응답
- [] UI 자연어: "피의자2의 계좌를 보여줘" → Cypher(vt_*) → AgensGraph → 결과
- [] 자연어 경로도 외부 호출 0건으로 완결

7.4 (선택) 8장 확장 메모

- 동시 사용자 처리량 ↑ → 카드당 1 vLLM 인스턴스 ×N + nginx 라운드로빈 (NVLink 무관, 선형 확장)
- 더 큰 모델(32B+) → 텐서병렬 대신 파이프라인 병렬(--pipeline-parallel-size)이 PCIe에서 유리

8. 롤백 / 트러블슈팅

증상	원인 후보	대응
1차: app 기동 실패	이미지 미적재, <code>.env</code> DB값	<code>docker images</code> 에 <code>ccop_app:1.0</code> 확인, <code>DB_HOST=agensgraph</code> · <code>DB_NAME=tccopdb</code>
1차: nginx 기동 실패	인증서 없음/경로 불일치	<code>deploy/ssl/cert.pem</code> · <code>key.pem</code> 존재 확인, 없으면 <code>gen_selfsigned_cert.sh</code>
1차: <code>/graph/list</code> 빈 그래프	DB 미복원, <code>DB_NAME=ccopdb</code>	<code>pg_restore</code> 재확인, <code>.env</code> 교정 후 <code>up -d app</code>
2차: <code>nvidia-smi</code> 실패	Secure Boot 미서명, 드라이버 미탑재	MOK 등록·재부팅, 드라이버 rpm 재설치
2차: vLLM 기동 실패	<code>VLLM_TAG</code> 부적합, 포트 점유	드라이버 CUDA와 태그 정합, <code>docker logs ccop_sllm</code>
2차: 자연어 질의 실패	<code>.env</code> SLLM 미반영, sllm 다운	<code>.env</code> 세 줄 확인 후 <code>up -d app</code> , sllm 로그

- **1차 롤백:** 문제 시 sllm 미기동 상태 유지(자연어만 미동작) — 인프라는 계속 사용 가능.
- **2차 롤백:** `.env` 의 SLLM 세 줄 주석 → `docker compose -f docker-compose.airgap.yml up -d app` → 1차 상태로 즉시 복귀 (sllm 중지: `docker stop ccop_sllm`).

9. 부록

9.1 번들 매니페스트 & 용량

단계	반입물	용량	USB
1차	app·agensgraph·nginx 이미지 + docker rpm + DB덤프 + 소스(airgap 자산·인증서 포함)	~4-5GB	8GB
2차	vLLM 이미지 + 모델 풀웨이트 + NVIDIA 드라이버/toolkit rpm	~30GB	64GB

9.2 핵심 주의 요약

- staging은 **linux/amd64** (Apple Silicon 빌드 금지).
- USB는 **exFAT** (FAT32 4GB 제한 회피).
- 반입 후 항상 `sha256sum -c`.
- 1차 `.env` 는 `SLLM_ENDPOINT` · `OPENAI_API_KEY` **미설정** → 앱은 자연어만 빼고 정상.
- Rocky 10 드라이버는 **CUDA repo 표준 rpm** (9/8의 dnf module과 다름).
- RTX 6000 Ada는 **NVLink 없음** → vLLM `--tensor-parallel-size 1` (8로 묶지 말 것).
- 1차↔2차 전환은 `.env` 세 줄 + `--profile gpu` + **app 재기동** (compose 편집·이미지 재빌드 없음).

9.3 폐쇄망 전용 자산 (이 저장소)

파일	설명
<code>docker-compose.airgap.yml</code>	app=사전이미지, sllm= <code>profile: gpu</code> (2차)
<code>deploy/.env.airgap.phase1.template</code>	1차 env 템플릿 (LLM 미설정)
<code>scripts/gen_selfsigned_cert.sh</code>	<code>cert.pem</code> / <code>key.pem</code> 생성

9.4 관련 문서

문서	내용
<code>AIRGAP_DEPLOY_GUIDE.md</code>	폐쇄망 설치 일반 원리·의존성 인벤토리
<code>VM_DEPLOY_OPERATIONS_GUIDE.md</code>	인터넷 운영 VM(skai2_vm) 배포·운영
<code>CHECKPOINT_20260616.md</code>	vLLM 서빙 재현(cu121·chat_template)·systemd 유닛
<code>../deploy/DEPLOY_CSLEE.md</code>	최초 배포 패키징·API 엔드포인트